SGD — Divisão de Tarefas do Grupo

22/10/2026 · @Vini

Visão geral

O SGD (Sistema de Gestão de Demandas) foi reduzido a duas telas: **Login** e **Demandas** (CRUD). O login será integrado à API da faculdade, que será enviada depois; até lá usamos um *mock*. **Somente usuários ADMIN** criam, editam e excluem demandas. Qualquer usuário autenticado consulta. Todo erro da API sai num formato único, por um **tratamento global de erros**.

Cada integrante fica com uma fatia do back-end, uma fatia do front-end, os cinco tipos de teste e uma tarefa de infraestrutura.

> **Atenção 1:** a orientação de avaliação diz "grupos de 2 até 4 integrantes (no máximo 4 pessoas)". O grupo tem 5. Confirmar com o professor antes de começar.
> **Atenção 2:** o projeto usa **Java 17**. Todos devem ter o JDK 17 instalado (veja o guia no fim de cada PDF).

Stack

Camada	Tecnologia
Back-end	Spring Boot 3 + Java 17, Maven, camadas Controller → Service → Repository
Banco	PostgreSQL em Docker, migrations com Flyway
Segurança	Spring Security + JWT, senhas fora do código; perfis `ADMIN` e `ALUNO`
Integração externa	Spring Cloud OpenFeign (API da faculdade; mock até ela chegar)
Documentação da API	Swagger UI (springdoc-openapi)
Front-end	Angular 19, MDB-Angular, SweetAlert2, modais, rotas filhas, services e models em TypeScript
Testes	JUnit 5, Mockito, MockMvc, Testcontainers, JaCoCo, JMeter, SonarQube

Estágio esperado (Trabalho Anterior)

A atividade anterior pedia este conjunto de itens, que deveria estar pronto. Como ainda não está, ele faz parte da entrega. Cada item abaixo tem um dono.

Critério do trabalho anterior	Onde aparece no projeto	Dono
Padrão em camadas e Clean Code	Pacotes `controller`, `service`, `repository`, `entity`, `dto`	Todos
Verbos e status HTTP; `@PathVariable`, `@RequestParam`, `@RequestBody`	Endpoints da tabela "Endpoints"	Todos (cada um nos seus)

Critério do trabalho anterior	Onde aparece no projeto	Dono
DTOs e Bean Validation (`@Valid`)	`DemandaRequest`, `LoginRequest`	Integrantes 1 e 2
Integração externa com OpenFeign	`FaculdadeFeignClient`	Integrante 4
Swagger UI	`OpenApiConfig`	Integrante 5
Entidades JPA e relacionamentos	`Demanda` `@ManyToOne` `Empresa` (e `Empresa` `@OneToMany` `Demanda`)	Integrantes 2 e 3
Flyway	`V1__create_tables.sql`, `V2__insert_empresas.sql`	Integrantes 2 e 3
`@Transactional` e `@RestControllerAdvice`	Services e `GlobalExceptionHandler`	Integrantes 2, 5 e 4
Logs com SLF4J	Logs informativos e de erro nos pontos críticos dos services	Todos (padrão do Integrante 4)
Modais, MDB-Angular, SweetAlert2, Angular 19	Telas de Login e Demandas	Todos
Login, services, models, HTTP com tratamento de erro, rotas filhas	Veja a divisão do front-end abaixo	Todos
Todos contribuem no front-end (sem commits triviais)	Cada integrante tem pelo menos um componente ou service	Todos

Endpoints

Método e rota	Quem acessa	Respostas
`POST /api/auth/login`	Público	200 (token), 400, 401, 503
`GET /api/empresas`	Autenticado	200, 401
`GET /api/demandas?titulo=&situacao=&empresaId=&page=&size=`	Autenticado	200, 401
`GET /api/demandas/{id}`	Autenticado	200, 401, 404
`POST /api/demandas`	**ADMIN**	201, 400, 401, 403, 404, 409
`PUT /api/demandas/{id}`	**ADMIN**	200, 400, 401, 403, 404, 409
`DELETE /api/demandas/{id}`	**ADMIN**	204, 401, 403, 404

Formato único de erro (ErrorResponse)

```
{
  "timestamp": "2026-10-26T10:00:00",
  "status": 404,
  "error": "Not Found",
  "message": "Demanda 15 não encontrada",
  "path": "/api/demandas/15",
  "details": []
}
```

`details` traz a lista de campos inválidos nos erros 400 de validação. O corpo nunca traz *stack trace*.

Exceção (pacote `exception`)	Status
`RecursoNaoEncontradoException`	404
`RegraNegocioException`	400
`ConflitoException` e `ObjectOptimisticLockingFailureException`	409
`CredenciaisInvalidasException` e `AuthenticationException`	401
`AccessDeniedException`	403
`ServicoExternoIndisponivelException`	503
`MethodArgumentNotValidException` e JSON malformado	400
Qualquer outra (`Exception`)	500, mensagem genérica; o detalhe vai só para o log

Regras de negócio

Código	Regra
RN-01	Somente ADMIN cria, edita e exclui demandas. ALUNO e visitante só consultam (403 / 401).
RN-02	A demanda pertence a uma empresa existente. Título único dentro da mesma empresa.
RN-03	Situações da demanda: `DISPONIVEL` (padrão) e `ENCERRADA`. Demanda encerrada não pode ser editada.
RN-04	Edição com controle de versão: se duas pessoas editam a mesma demanda ao mesmo tempo, só a primeira grava e a outra recebe 409.
RN-05	O perfil do usuário vem da API da faculdade (ADMIN ou ALUNO) e vai dentro do token JWT.

Nomenclatura (padrão que aprendemos na faculdade)

Termos técnicos em inglês (Controller, Service, Repository, Entity, DTO, Request, Response). Nomes do domínio em português (Demanda, Empresa, situacao). Estrutura por camada técnica. Pacote base: `br.com.uniamerica.sgd` (combinar o nome definitivo com o Integrante 1).

O que	Padrão	Exemplo
Pacotes	minúsculos, um por camada (o repositório de exemplo usa maiúscula, mas a convenção Java e o SonarQube pedem minúscula)	`controller`, `service`, `repository`, `entity`, `dto`, `security`, `exception`, `config`, `client`
Entidade	Substantivo no singular	`Demanda`, `Empresa`
Controller / Service / Repository	Entidade + camada	`DemandaController`, `DemandaService`, `DemandaRepository`
DTOs	Entidade + Request ou Response	`DemandaRequest`, `DemandaResponse`, `LoginRequest`, `LoginResponse`
Enums	Substantivo	`SituacaoDemanda`, `Perfil`
Exceções	Descrição + Exception	`RecursoNaoEncontradoException`
Métodos de negócio	Verbo em português, camelCase	`listar`, `buscarPorId`, `criar`, `atualizar`, `excluir`
Tabelas e colunas	snake_case, singular	`demanda`, `empresa_id`
Migrations	`V{n}__{acao}_{assunto}.sql`	`V1__create_tables.sql`
Testes	Classe testada + Test (unitário) ou IT (integração)	`DemandaServiceTest`, `DemandaControllerIT`
Angular	kebab-case com sufixo do tipo	`demanda-lista.component.ts`, `demanda.service.ts`, `demanda.model.ts`, `auth.guard.ts`
Branches	`feature/` + assunto	`feature/auth-login`, `feature/demanda-listagem`

Divisão por integrante

Integrante	Módulo	Back-end	Front-end	Tarefa de infraestrutura
1	Login e segurança	`AuthController`, `AuthService`, `JwtUtil`, `JwtAuthFilter`, `SecurityConfig`	Tela de login, `auth.service`, `auth.guard`, `jwt.interceptor`	Repositório, esqueleto (Spring + Angular + Flyway + Postgres em Docker), `.gitignore`, README, branches

Integrante	Módulo	Back-end	Front-end	Tarefa de infraestrutura
2	Demandas: criar e atualizar	`Demanda`, `DemandaRepository`, `DemandaService` (criar, atualizar), `DemandaController` (POST, PUT), DTOs, `V1__create_tables.sql`	Models, `demanda.service` (HTTP), modal de cadastro e edição	JaCoCo no `pom.xml` com meta de 90% e relatório final
3	Empresas e estrutura do front	`Empresa`, `EmpresaRepository`, `EmpresaService`, `EmpresaController`, `V2__insert_empresas.sql`	Layout com navbar (MDB), `app.routes.ts` com rotas filhas, `empresa.service`	SonarQube em Docker e relatório da 1ª análise
4	Erros globais e API da faculdade	`GlobalExceptionHandler`, `ErrorResponse`, exceções, `FaculdadeFeignClient`, `FaculdadeGateway` (Feign e mock), padrão de logs	`error.interceptor`, `notificacao.service` (SweetAlert2), página 404	Plano base do JMeter (`.jmx`) e análise dos resultados
5	Demandas: consultar, excluir e concorrência	`DemandaService` (listar, buscarPorId, excluir), `DemandaController` (GET, DELETE), `OpenApiConfig`, controle de versão (RN-04)	Página de listagem (tabela, filtros, paginação), confirmação de exclusão, botões só para ADMIN	Relatório da 2ª análise do SonarQube e ZIP final

Quem abre o PR revisa o do próximo em rodízio: 1→2, 2→3, 3→4, 4→5, 5→1.

> **Classes compartilhadas:** o Integrante 2 cria `Demanda`, `DemandaService` e `DemandaController`. O Integrante 5 acrescenta os métodos dele nas mesmas classes **em PR separado**, depois que o PR do Integrante 2 entrar no `develop`. O mesmo vale para `app.routes.ts` (Integrante 3 cria, os outros só acrescentam rotas).

Contratos entre os integrantes

Combinar estes pontos na primeira semana, para ninguém ficar bloqueado:

- **Token:** cabeçalho `Authorization: Bearer <token>`. O token traz o login (`sub`), o nome e o perfil (`ADMIN` ou `ALUNO`).
- **API da faculdade (ainda desconhecida):** o Integrante 4 cria a interface `FaculdadeGateway` com um método `autenticar(login, senha)` que devolve nome e perfil. Hoje responde o mock (perfil Spring `mock`). Quando a API chegar, só a implementação Feign muda.
- **Exceções e `ErrorResponse`:** o Integrante 4 entrega primeiro (semana 1), porque todos lançam essas exceções.
- **Migrations:** o número da migration é único. Integrante 2 usa `V1`, Integrante 3 usa `V2`. Qualquer outra migration: avisar o grupo antes.
- **Lombok:** arquivo `lombok.config` na raiz com `lombok.addLombokGeneratedAnnotation = true`, para o JaCoCo ignorar o código gerado.

Ordem de trabalho

1. **Semana 1:** Integrante 1 entrega esqueleto, Docker e login com mock. Integrante 4 entrega exceções e `GlobalExceptionHandler`. Integrante 2 entrega entidade e migration. Integrante 3 entrega layout e rotas.
2. **Semanas 2 e 3:** cada um implementa a sua fatia com **os testes no mesmo PR do código**.
3. **SonarQube, 1ª análise:** com todos os módulos no `develop`, sem corrigir nada antes (Integrante 3).
4. **Correções:** cada um corrige o seu módulo. Depois, 2ª análise (Integrante 5), relatório final do JaCoCo (Integrante 2) e JMeter completo (Integrante 4).
5. **Entrega:** ZIP sem `target/`, `build/` e `node_modules/`, mais o link do repositório.

Os prazos de liberação do Projeto de Software eram 06/11/2026 e 27/11/2026. Ajustar à data definida pelo professor.

Testes: todos fazem todos

Cada integrante faz os cinco tipos de teste no próprio módulo. Se cada módulo bater 90%, o projeto bate 90%. O passo a passo de cada tipo está no **fim do PDF de cada integrante**.

Plataforma	O que cada integrante faz
JUnit 5 + Mockito	Testes unitários dos `*Service` e das classes com lógica, com `@Mock`, `@InjectMocks`, `when/thenReturn` e `verify`, com cenário de sucesso e de erro. Não testar getters, DTOs nem repositories.
Integração	`@SpringBootTest` + MockMvc + Testcontainers (PostgreSQL). Testar os endpoints do próprio módulo de ponta a ponta, incluindo a persistência.
JaCoCo	≥ 90% nos pacotes e classes do próprio módulo. Excluir da contagem só `Application`, configurações e DTOs.
JMeter	1 Thread Group no `.jmx` compartilhado, com o cenário abaixo.
SonarQube	Na 1ª análise, explicar os problemas do próprio módulo. Depois corrigir e mostrar a evolução na 2ª.

Cenários do JMeter (um por integrante)

Integrante	Cenário	Meta
1	Login de 100 usuários	Resposta ≤ 3 s, 0% de erro
2	ADMINs criando e atualizando demandas	Sem erros sob carga
3	Listagem de empresas com 200 usuários	Resposta ≤ 3 s
4	Requisições inválidas e sem token	Só 4xx esperados, nenhum 500
5	Vários ADMINs editando a mesma demanda	Exatamente 1 edição vence, as outras recebem 409

Regras do GitHub

O uso do GitHub vale 1,0 ponto e o professor confere a participação de cada um pelo histórico de commits.

- Branches: `main` (entrega), `develop` (integração) e uma `feature/` por entrega.
- Pull request obrigatório, revisado por outra pessoa em rodízio.
- Commits pequenos e frequentes, no padrão Conventional Commits, em português: `feat`, `test`, `fix`, `refactor`, `docs`, `chore`.
- Todo mundo commita código e testes. Todo mundo commita no front-end.
- Nunca commitar token do SonarQube, senhas reais, `.idea/`, `target/` nem `node_modules/`.

Tipo	Exemplo correto
feat	`feat(demanda): cria endpoint de cadastro restrito a ADMIN`
test	`test(demanda): cobre exclusão de demanda inexistente`
fix	`fix(sonar): remove senha fixa da configuração do mock`
refactor	`refactor(erro): extrai montagem do ErrorResponse`
docs	`docs: adiciona instruções de execução ao README`
chore	`chore: configura plugin do JaCoCo no pom.xml`

Proibido: "bug dos infernos", "agora vai", "ajustes", "asdf" ou qualquer mensagem que não diga o que mudou.

Checklist de entrega

☐ Repositório com README, `.gitignore` e commits de todos (Integrante 1)
☐ Relatório do JaCoCo com cobertura ≥ 90%, pasta ou print do `index.html` (Integrante 2)
☐ Testes unitários (JUnit + Mockito) e de integração no repositório (todos)
☐ Relatório da 1ª análise do SonarQube com os problemas explicados (Integrante 3)
☐ Plano JMeter (`.jmx`) e resultados com análise de tempo de resposta, erros e gargalos (Integrante 4)
☐ Relatório da 2ª análise do SonarQube mostrando as correções (Integrante 5)
☐ Arquivo `.zip` sem `target/`, `build/` e `node_modules/`, igual ao repositório (Integrante 5)
☐ Link do repositório, público ou com acesso ao professor (Integrante 1)